README - Tugas Praktikum P6

Pemrograman Berorientasi Obyek — Polymorphism, Method Overriding, Method Overloading, dan Dynamic Binding

Nama Mahasiswa	: Aldo Rachmad Hidayat
NRP	: 3125522003
Program Studi	: D3 Teknik Informatika - PENS PSDKU Sumenep

1. Nama Proyek

SiKos - Sistem Informasi Manajemen Kos

2. Sprint Goal (P6)

Mengembangkan behavior object pada proyek melalui method overriding dan polymorphism sehingga subclass Penyewa dan PemilikKos dapat merespons pemanggilan method yang sama dengan perilaku berbeda.

3. Sprint Backlog

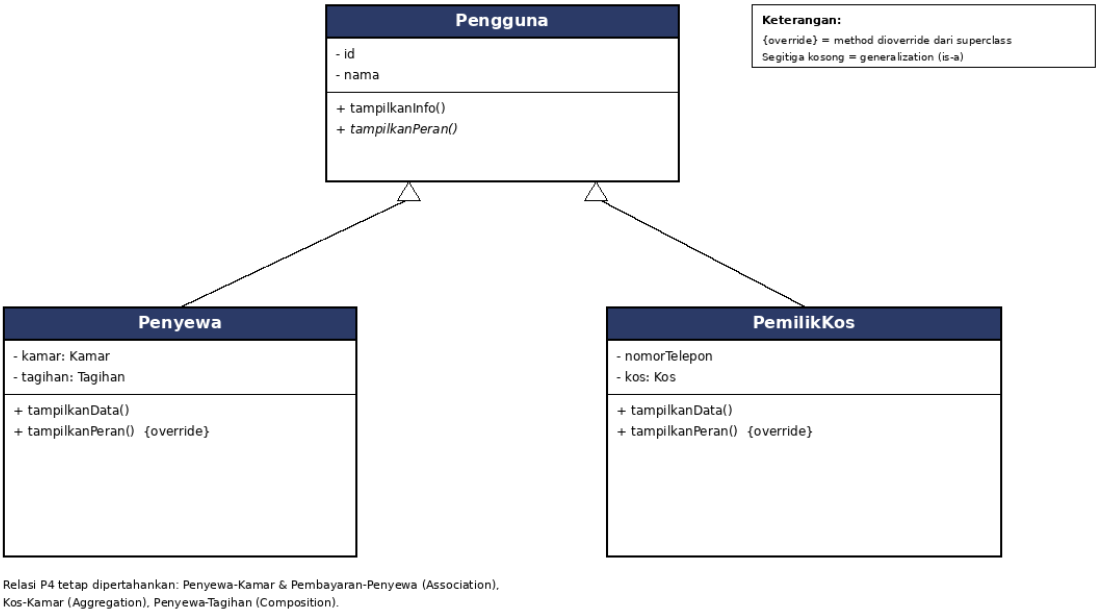
ID	Sprint Backlog	Status
SB-01	Audit superclass dan subclass hasil P5	Done
SB-02	Menentukan behavior yang dapat dioverride	Done
SB-03	Implementasi overriding (tampilkanPeran)	Done
SB-04	Implementasi overloading sederhana (tambahKamar)	Done
SB-05	Implementasi upcasting	Done
SB-06	Membuat polymorphic collection	Done
SB-07	Melakukan pengujian dynamic binding	Done

4. Hierarchy Class (hasil P5) & Behavior Terpilih

Hierarchy yang digunakan: **Pengguna** (superclass) → **Penyewa**, **PemilikKos** (subclass). Behavior yang dipilih untuk polymorphism adalah **tampilkanPeran()** — method ini ada di superclass Pengguna, namun setiap subclass memberikan implementasi yang berbeda sesuai perannya masing-masing.

Superclass	Subclass	Method	Implementasi Berbeda
Pengguna	Penyewa	tampilkanPeran()	"Penyewa - menempati kamar dan membayar tagihan sewa"
Pengguna	PemilikKos	tampilkanPeran()	"Pemilik Kos - mengelola data kamar dan memantau penyewa"

5. Class Diagram Terbaru



6. Contoh Overriding

```
public class Penyewa extends Pengguna {
  @Override
  public void tampilkanPeran() {
    System.out.println("Penyewa - menempati kamar...");
  }
}
```

7. Contoh Overloading

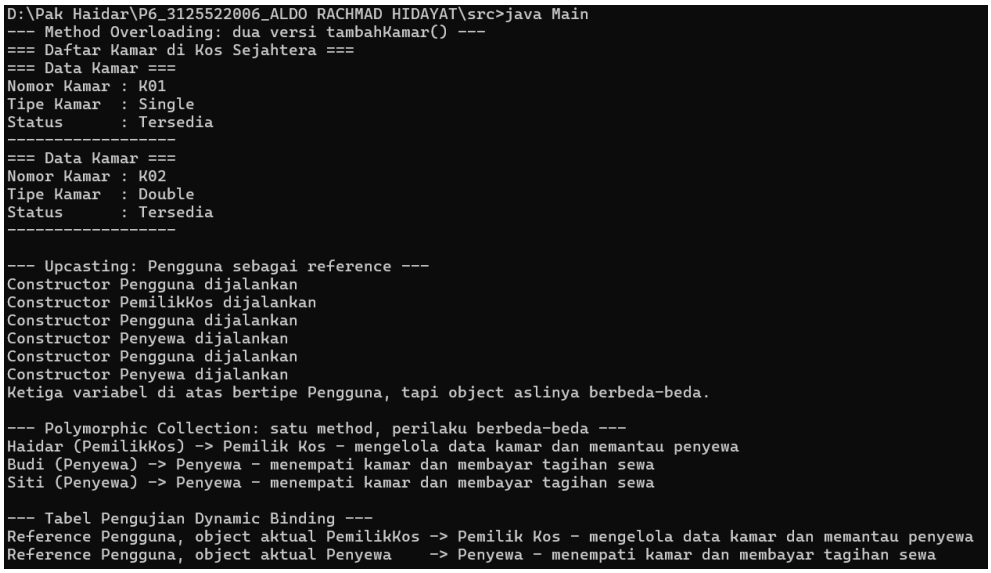
Class Kos memiliki dua method bernama sama `tambahKamar()` dengan parameter berbeda — satu menerima object Kamar yang sudah jadi, satu lagi menerima data mentah dan membuat object Kamar-nya sendiri:

```
public void tambahKamar(Kamar kamar) { ... }
public void tambahKamar(String nomorKamar, String tipeKamar, boolean statusTersedia) {
  Kamar kamarBaru = new Kamar(nomorKamar, tipeKamar, statusTersedia);
  daftarKamar.add(kamarBaru);
}
```

8. Penjelasan Upcasting

Object Penyewa dan PemilikKos dibuat lewat reference bertipe superclass-nya: `Pengguna pengguna1 = new PemilikKos(...);` dan `Pengguna pengguna2 = new Penyewa(...);`. Walaupun tipe reference-nya sama-sama Pengguna, object yang sebenarnya berbeda, sehingga saat method tampilkanPeran() dipanggil, Java menjalankan implementasi milik object aslinya (dynamic binding) — inilah yang memungkinkan polymorphism.

9. Screenshot Hasil Running



```
D:\Pak Haidar\P6_3125522006_ALDO RACHMAD HIDAYAT\src>java Main
--- Method Overloading: dua versi tambahKamar() ---
=== Daftar Kamar di Kos Sejahtera ===
=== Data Kamar ===
Nomor Kamar : K01
Tipe Kamar  : Single
Status      : Tersedia
------------------
=== Data Kamar ===
Nomor Kamar : K02
Tipe Kamar  : Double
Status      : Tersedia
------------------

--- Upcasting: Pengguna sebagai reference ---
Constructor Pengguna dijalankan
Constructor PemilikKos dijalankan
Constructor Pengguna dijalankan
Constructor Penyewa dijalankan
Constructor Pengguna dijalankan
Constructor Penyewa dijalankan
Ketiga variabel di atas bertipe Pengguna, tapi object aslinya berbeda-beda.

--- Polymorphic Collection: satu method, perilaku berbeda-beda ---
Haidar (PemilikKos) -> Pemilik Kos - mengelola data kamar dan memantau penyewa
Budi (Penyewa) -> Penyewa - menempati kamar dan membayar tagihan sewa
Siti (Penyewa) -> Penyewa - menempati kamar dan membayar tagihan sewa

--- Tabel Pengujian Dynamic Binding ---
Reference Pengguna, object aktual PemilikKos -> Pemilik Kos - mengelola data kamar dan memantau penyewa
Reference Pengguna, object aktual Penyewa    -> Penyewa - menempati kamar dan membayar tagihan sewa
```

10. Tabel Pengujian Dynamic Binding

Reference	Object Aktual	Method	Output
Pengguna	PemilikKos	tampilkanPeran()	Pemilik Kos - mengelola data kamar dan memantau penyewa
Pengguna	Penyewa	tampilkanPeran()	Penyewa - menempati kamar dan membayar tagihan sewa

Method yang dijalankan berbeda walaupun reference sama-sama bertipe Pengguna, karena Java menentukan method yang dipanggil berdasarkan object aktual di memori (dynamic binding), bukan berdasarkan tipe reference-nya.

11. Sprint Review

Item	Hasil
Superclass dan subclass tersedia	Ya, Pengguna sebagai superclass, Penyewa & PemilikKos sebagai subclass
Method overriding berhasil	Ya, tampilkanPeran() di-override di 2 subclass dengan @Override
Minimal 2 subclass beda behavior	Ya, Penyewa dan PemilikKos menampilkan pesan berbeda
Method overloading tersedia	Ya, tambahKamar() punya 2 versi parameter berbeda di class Kos
Upcasting berhasil	Ya, variabel bertipe Pengguna menampung object Penyewa/PemilikKos
Polymorphic collection berhasil	Ya, array Pengguna[] berisi 3 object dari 2 subclass berbeda
Dynamic binding dapat dibuktikan	Ya, output tampilkanPeran() berbeda sesuai object asli
Program berjalan	Ya, program berhasil dikompilasi dan dijalankan tanpa error
Kendala	Perlu memastikan overloading yang dibuat relevan, bukan sekadar contoh formalitas

12. Sprint Retrospective

What Went Well? Overriding tampilkanPeran() berhasil menunjukkan polymorphic behavior yang jelas antara Penyewa dan PemilikKos, dibuktikan lewat polymorphic collection dan dynamic binding.

What Went Wrong? Sempat bingung membedakan overloading dan overriding karena keduanya sama-sama menggunakan nama method yang sama; kuncinya ada pada parameter dan hubungan class.

Improvement Pada sprint berikutnya (P7) akan dievaluasi apakah class Pengguna sebaiknya menjadi abstract class, karena tidak seharusnya dibuat object secara langsung tanpa menjadi Penyewa atau PemilikKos.